Event-Triggered Polynomial Control

Trajectory Tracking for Unicycle Robots

1 Introduction

This paper focuses on designing an intelligent controller for mobile robots designed to:

- ✓ Efficiently **track a path**.
- ✓ Minimize **network communication** overhead.
- ✓ Preserve high **control performance**.

This is critical for **Networked Control Systems (NCS)**.

The Objective

Trajectory tracking ensures the robot's state $(x(t), y(t))$ aligns with the desired path $(x_r(t), y_r(t))$.

$$\text{actual path} \approx \text{desired path}$$

2 Why is This Hard?

Controller and robot are often decoupled by a communication layer:

$$\boxed{\text{Controller}} \longleftrightarrow \boxed{\text{Network}} \longleftrightarrow \boxed{\text{Robot}}$$

Challenges

1. **High Cost:** Communication bandwidth is often limited.
2. **Resource Depletion:** Constant updates drain battery.

3 Traditional Solution

Time-Triggered Control:

- $t = 0\,\text{ms} \longrightarrow$ send control
- $t = 10\,\text{ms} \longrightarrow$ send control

4 Event-Triggered Control (ETC)

In Time-Triggered Control (TTC), signals are sent periodically (e.g., every 10ms). This often results in communication even when no update is necessary, wasting **power, CPU, and bandwidth**.

To solve this, we move to **Event-Triggered Control**:

*Send control signals **only when needed**.*

Logic Comparison

Small Deviation $\longrightarrow$ Do nothing (save resources)

Large Deviation $\longrightarrow$ Send update (trigger event)

Advantages over TTC:

- ✓ Less communication overhead
- ✓ Maintains the same performance levels
- ✓ Significantly more efficient resource usage

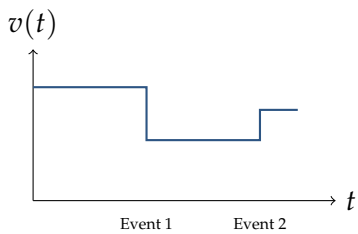
5 Limitations of Existing ETC

Despite its efficiency, most standard ETC systems rely on a mechanism that creates a bottleneck:

Zero Order Hold (ZOH)

When the controller sends a value, the actuator **holds it constant** until the next update is received. This behaves like a *step signal*.

Example of ZOH behavior: If the controller sends a velocity command $v = 2$ m/s, the robot applies exactly 2 m/s as a flat constant until the next "event" triggers a change.



This "staircase" approximation can be inefficient for complex trajectory tracking where smooth, polynomial changes are preferred.

The Problem with ZOH

As previously noted, ZOH produces a **step signal**. Because one communication packet typically contains only one numerical value, the control signal between updates remains strictly **constant**.

However, the real optimal control signal for tracking a smooth trajectory often needs to be **changing continuously**. By sending a constant value, the available communication bandwidth is effectively *under-utilized*.

Stop and Ponder

Why do we want a changing control signal? If a change is needed, isn't that just another event?

ANSWER: In Event-Triggered Control (ETC), an event is **NOT** defined as "any time the control signal should change." Instead, an event is triggered only when:

The system error has become large enough that the controller must recompute and transmit new control laws.

Error-Based Events: Events depend on the measured **error**, not simply on whether the control signal is changing. Mathematically, an event occurs when the tracking error $e(t)$ exceeds a specific threshold defined by a triggering function:

$$\|e(t)\| > \sigma\|x(t)\| + \delta$$

where:

- $e(t)$ is the difference between the actual and desired state.
- σ and δ are design parameters determining the sensitivity of the trigger.

This is why **polynomial control** is advantageous; it allows the control signal to evolve intelligently between these error-triggered communication events, rather than staying stuck at a single constant value.

6 The Core Idea: Polynomial Event-Triggered Control

In physical systems, the ideal control signal is usually **continuous**. For example, following a curved path requires continuous adjustments. Forcing the control to be constant (via ZOH) inherently makes the motion less accurate.

Why not trigger an event every time the control needs to change? If we triggered an event for every change in an ideal signal like $u(t) = \sin(t)$, we would need frequent updates to maintain a piecewise constant approximation. Communication would effectively *"explode,"* defeating the purpose of ETC.

The Solution: Polynomial Functions

The real goal of ETC is to **minimize communication while keeping error small**. Instead of sending a single constant value, we send a **polynomial function of time**. The robot then computes its own control locally based on this function.

Comparison of Control Packets

⇒ **Traditional ETC (ZOH):**
The packet sends a constant:

$$v = 2$$

⇒ **Proposed Polynomial ETC:**
The packet sends coefficients for a function:

$$v(t) = 2 + 0.4t - 0.1t^2$$

The Result: Because a polynomial can "predict" the required change in control over a short duration, one single communication packet can control the robot accurately for a **significantly longer time interval**. This drastically reduces the number of events (transmissions) required to maintain high tracking precision.

7 The Outcome of Polynomial Control

By utilizing polynomials, the communication frequency becomes **much smaller**. This results in significant savings in **CPU power** and **network bandwidth**, while enabling more **agile motion**.

The actuator can now generate smooth control locally. The robot follows a changing control trajectory without needing to query the controller repeatedly, effectively **reducing the number of events**.

The Engineering Trade-off

This paper explores the trade-off between **communication cost** and **computation cost**. The primary focus is on drastically reducing the communication cost, which is the main bottleneck in networked systems.

What Exactly Do They Propose?

The authors propose a method called **Event-Triggered Polynomial Control (ETPC)**.

7]

When an event is triggered by the error threshold...

The controller transmits **polynomial coefficients** rather than a single value.

The robot then applies the following control law locally:

$$u(t) = a_0 + a_1 t + a_2 t^2 + \dots$$

This continues until the **next event** is triggered.

8 Key Guarantees of the Paper

The significance of this work lies in its theoretical rigor. The authors mathematically prove that under this ETPC scheme:

- ✓ **Tracking error stays bounded:** The robot will never drift infinitely far from the path.

Formal Stability: This property is formally referred to as **Uniform Ultimate Boundedness (UUB)**. It guarantees that the system state will eventually enter and remain within a specific small region around the desired trajectory.

No Zeno Behavior: A critical guarantee in event-triggered systems is the absence of "Zeno" behavior. This occurs when a controller attempts to send an infinite number of updates in a finite amount of time, which would crash the system.

2]

Non-Zeno: The system is mathematically guaranteed to have a minimum positive time interval between any two consecutive updates.

9 The Unicycle Robot Model

The paper applies this ETPC method specifically to the **Unicycle Robot** model. This robot moves like a single wheel; it can move forward or backward and rotate, but it cannot move sideways (known as a *non-holonomic* constraint).

Kinematic Equations

The motion of the robot is modeled by the following differential equations:

$$\dot{x} = v\cos\theta$$
$$\dot{y} = v\sin\theta$$
$$\dot{\theta} = \omega$$

Variable Definitions

Variable	Meaning
x, y	Cartesian position
θ	Orientation (heading angle)
v	Linear velocity (control input)
ω	Angular velocity (control input)

Conclusion: By combining the **Unicycle Model** with **Event-Triggered Polynomial Control**, the authors provide a framework that ensures the robot stays within a stable "box" around the desired trajectory (**UUB**) while significantly lowering the communication burden on the network.

10 Reference Trajectory

To "track" a trajectory, the robot needs a target to follow, often referred to as a "ghost" or **Reference Trajectory**. This reference follows the same mathematical model as the robot but uses reference variables: $(x_r, y_r, \theta_r, v_r, \omega_r)$.

Important Assumption (A1)

The paper assumes that the reference velocities and their derivatives are bounded:

$$|v_r|, |\omega_r|, |\dot{v}_r|, |\dot{\omega}_r| \text{ are bounded}$$

AND

$$|v_r| \geq c > 0$$

Intuition:

10]

Smoothness: The reference path must be smooth.

Persistence: The robot is always moving (non-zero velocity) to avoid mathematical singularities and simplify control design.

11 Tracking Error: A Smarter Definition

The goal of the controller is to make:

$$\text{robot state} \approx \text{reference state}$$

While a simple subtraction (reference − actual) seems intuitive, this paper uses a **Rotation Matrix** to define the error in the **Robot's Frame** rather than the Global Frame.

The Transformation Matrix

$$\begin{bmatrix} x_e \\ y_e \\ \theta_e \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_r - x \\ y_r - y \\ \theta_r - \theta \end{bmatrix}$$

Breaking down the rows:

- **First Row (x_e):** Longitudinal Error. Tells the robot how far "forward" or "backward" the target is relative to its own nose.
- **Second Row (y_e):** Lateral Error. Tells the robot how far to the "left" or "right" the target is.
- **Third Row (θ_e):** Heading Error. A simple angular subtraction ($\theta_r - \theta$).

Why use the Robot Frame?

Imagine your GPS says "Your destination is 50 meters North."

- **Scenario A:** You are facing North. You simply drive straight.
- **Scenario B:** You are facing South. You must turn 180° first.

By defining error in the **Robot Frame**, the controller knows exactly what physical action (steering or accelerating) is needed based on the robot's current orientation.

12 Why the Robot Frame is Better for Engineers

In both the Global and Robot frames, the numerical "Global Error" ($x_r - x$) might be identical. However, the required **control action** (whether to drive forward or turn) is completely different based on orientation. By calculating error in the **Robot Frame**, we translate the global world into the robot's perspective: *"Is the target in front of me, to my side, or behind me?"*

Key Advantage

Using the **robot frame** simplifies the control logic by aligning mathematical errors directly with the robot's physical capabilities.

1. Direct Control Mapping

In the global frame, reducing an error in the "X" direction might require both driving and turning simultaneously. In the **Robot Frame**, the error is naturally decoupled:

- **Longitudinal Error (x_e):** Primarily fixed by adjusting the **linear velocity** (v).
- **Lateral (y_e) and Heading (θ_e) Errors:** Primarily fixed by adjusting the **angular velocity** (ω).

2. Handling Non-Holonomic Constraints

A unicycle robot is **non-holonomic**, meaning it cannot move sideways instantly (unlike a person sidestepping).

Comparison:

- **Global Frame:** The math must constantly account for the fact that the robot cannot simply move toward a coordinate (X, Y) without turning first.
- **Robot Frame:** The lateral error (y_e) explicitly tells the controller: *"You are off-track to the side; you must apply angular velocity (ω) to fix this."*

13 Kinematic Modeling of Error

The tracking error in the robot frame is defined by the following transformation:

$$\begin{bmatrix} x_e \\ y_e \\ \theta_e \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_r - x \\ y_r - y \\ \theta_r - \theta \end{bmatrix}$$

To simplify the notation, we let:

$$\Delta x = x_r - x$$

$$\Delta y = y_r - y$$

Which gives us the individual error components:

$$\begin{aligned} x_e &= \Delta x \cos\theta + \Delta y \sin\theta \\ y_e &= -\Delta x \sin\theta + \Delta y \cos\theta \\ \theta_e &= \theta_r - \theta \end{aligned}$$

Evolution of Tracking Error

To design an effective controller, we must understand the **derivative** of the error (how it changes over time). Our goal is to design a controller that makes the derivative of the error negative when the error itself is positive, driving the system toward zero error.

Deriving $\dot{x}_e$: Using the **chain rule**, we calculate the derivative of the longitudinal error:

$$\dot{x}_e = \frac{d}{dt}(\Delta x \cos\theta + \Delta y \sin\theta)$$

Expanding this via the product and chain rules:

$$\dot{x}_e = (\dot{\Delta x} \cos\theta - \Delta x \dot{\theta} \sin\theta) + (\dot{\Delta y} \sin\theta + \Delta y \dot{\theta} \cos\theta)$$

Controller Objective

The ultimate aim of these kinematics is to find a control law for v and ω such that the derivatives $\dot{x}_e, \dot{y}_e$, and $\dot{\theta}_e$ are forced to converge to zero.

14 Deriving the Full Error Dynamics

By substituting the robot's dynamics $(\dot{x}, \dot{y}, \dot{\theta})$ into the derivative of the error vector, we obtain the time-evolution of the tracking error in the robot frame:

Kinematic Error Model

$$\begin{aligned}
\dot{x}_e &= v_r \cos\theta_e - v + \omega y_e \\
\dot{y}_e &= v_r \sin\theta_e - \omega x_e \\
\dot{\theta}_e &= \omega_r - \omega
\end{aligned}$$

15 State-Space Decomposition

To facilitate control design, we group these results into a standard state-space form. We define the error state vector as $X = [x_e, y_e, \theta_e]^T$. The system is then decomposed into a **"Free Flow"** part (F) and a **"Control"** part (G):

$$\dot{X} = F(X,t) + G(X)U$$

Where the control input is $U = [v, \omega]^T$. Expanding the matrices gives:

$$\begin{bmatrix} \dot{x}_e \\ \dot{y}_e \\ \dot{\theta}_e \end{bmatrix} = \underbrace{\begin{bmatrix} v_r \cos\theta_e \\ v_r \sin\theta_e \\ \omega_r \end{bmatrix}}_{F(X,t)} + \underbrace{\begin{bmatrix} -1 & y_e \\ 0 & -x_e \\ 0 & -1 \end{bmatrix}}_{G(X)} \begin{bmatrix} v \\ \omega \end{bmatrix}$$

Understanding the Components

- **Free Flow ($F(X,t)$):** This represents what happens to the error if the robot just sits still ($v = 0, \omega = 0$). The error changes simply because the **target is moving away**.
- **Control Part ($G(X)$):** This represents the "Control Effort." It shows how our control inputs v and ω "fight back" against the growing error to maintain the trajectory.

Why this structure? This specific decomposition allows the designer to see exactly how linear and angular velocities influence each error component. For example, v only appears in the $\dot{x}_e$ equation, while ω influences all three components due to the robot's rotational geometry.

16 Standard Control Form: The Big Picture

In modern control theory, most complex systems can be decomposed into a fundamental relationship between how the world changes and how the controller reacts. This is expressed as:

Rate of Change = (What the world does) + (What YOU do)

Formally, this is the **Standard Nonlinear Control Form**:

$$\dot{X} = F(x,t) + G(x)U$$

- **State Derivative ($\dot{X}$):** How the error evolves.

- **System Part (F):** Internal dynamics/Reference movement.
- **Control Input (U):** The actions taken by the robot.

Why this matters: By structuring the robot's kinematics this way, we can apply advanced techniques like **Lyapunov Control** and **Optimization** to ensure the robot never leaves its intended path.

17 The Core Innovation: Non-ZOH Control Law

Traditional event-triggered systems use a *Zero-Order Hold* (ZOH), which sends a single value that stays constant. We propose a **Non-ZOH** law. Instead of a single number, the controller sends a **formula**.

This allows the robot to "predict" how it should move between messages without asking the controller for help. It effectively fills in the gaps between communication events.

The Polynomial Control Equation

Consider a polynomial control input whose coefficients are updated only at each discrete event t_k:

$$U_i(t_k + \tau) = \sum_{j=0}^{p} a_{ji}(k)\tau^j$$

Variable Breakdown:

- U_i**:** The control input (either linear velocity v or angular velocity ω).
- t_k**:** The exact moment an "event" happens and the controller sends a message.
- τ**:** The time elapsed since the last event.
- $a_{ji}(k)$**:** The coefficients transmitted by the controller.

The Result: By using this polynomial expansion, the robot follows a smooth, curved control trajectory that mimics the "ideal" continuous signal. This significantly delays the time it takes for the error to hit the threshold, thereby **drastically reducing the total number of transmissions**.

18 Local Time vs. Global Time

The key to implementing polynomial control is understanding the difference between absolute (global) time and the time elapsed since the last event.

- **Global Time (t):** The standard continuous time. Example: 1:00 PM.
- **Local Time (τ):** The time counter on the robot's onboard CPU, which is reset to zero every single time a new "event" (t_k) happens.

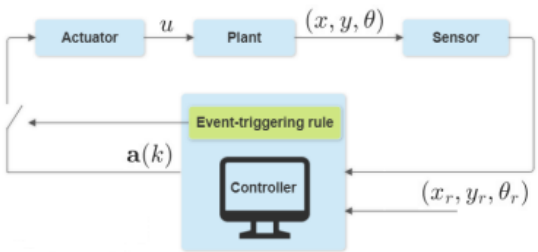


Fig. 1: Configuration of ETPC

Figure 1: Enter Caption

Example Calculation: Imagine the previous message was sent at $t_k = 1{:}00$ PM. If the current time is $t = 1{:}00{:}05$ PM, then the local time on the robot is:

$$\tau = t - t_k = 5 \text{ seconds}$$

This τ clock will immediately reset when the next message arrives.

The Effect of Polynomial Order (p)

The degree of the polynomial, defined as 'p', determines the complexity of the motion the robot can execute between updates without asking the controller for help.

Case 18]

Linear Control ($p = 1$):
The control input U (velocity or angular rate) is a simple linear function of τ. This forces the robot to move only in straight lines between messages.

$$U = a_0 + a_1\tau$$

Cubic Control ($p = 3$):
The control is a cubic function. This allows the robot to handle complex, smooth curved trajectories over a much longer time interval.

$$U = a_0 + a_1\tau + a_2\tau^2 + a_3\tau^3$$

NOTE: The Control Recipe

Instead of sending a continuous "speed" signal, the controller only sends the "recipe" $(a_0, a_1, a_2, \dots)$. The robot's onboard CPU plugs the current local time (τ) into this formula to calculate what the exact speed should be at any given millisecond.

19 System Configuration

The architecture below illustrates how the ETPC controller interacts with the robot via the network, utilizing the polynomial "recipe" and local time (τ) to maintain smooth motion.

Note: The robot's onboard CPU acts as the "Local Controller," intercepting the polynomial coefficients and generating the high-frequency control signals locally.

20 Design of Polynomial Control Law

To design an efficient control strategy, the approach begins with an existing **continuous-time nonlinear feedback controller**, given by:

$$\hat{u}(t) = \begin{bmatrix} \hat{v}(\hat{X}, t) \\ \hat{\omega}(\hat{X}, t) \end{bmatrix}$$

Designing such a controller from scratch is generally difficult. Therefore, instead of constructing a new controller, the idea is to take this **ideal continuous control law** and approximate it in a form that can be implemented efficiently in a networked setting.

Predicted State and Its Role

Since the controller does not continuously communicate with the robot, it does not have access to real-time state updates between communication events. To overcome this, the controller maintains a **predicted error state**:

$$\hat{X} = [\hat{x}_e, \hat{y}_e, \hat{\theta}_e]$$

This predicted state is not the actual state but an internally simulated estimate. The reason for this is that between events, the controller is not updated with new measurements, so it must rely on its internal model of the system.

At each event time t_k, the predicted state is reset to the actual state:

$$\hat{X}(t_k) = X(t_k)$$

After this, the predicted state evolves according to the same system dynamics:

$$\dot{\hat{X}} = F(\hat{X}, t) + G(\hat{X})\, \hat{u}(\hat{X}, t)$$

This represents the evolution of the system as a combination of:

- **Free flow** (natural system evolution)
- **Control action** (effect of inputs)

Core Idea: Polynomial Approximation of Control

The central idea of the method is to approximate the continuous-time control signal using a polynomial function over time. That is:

$$\text{Polynomial control} \approx \text{continuous control}$$

Instead of transmitting instantaneous control values, the controller computes a polynomial:

$$u(t) = a_0 + a_1 t + a_2 t^2 + \cdots + a_p t^p$$

which approximates the ideal control signal:

$$u(t) \approx \hat{u}(t)$$

This polynomial is then used by the robot to compute control inputs locally over time.

21 Optimization-Based Coefficient Design

To determine the coefficients of the polynomial, an optimization problem is solved at each event time t_k. The objective is to minimize the difference between the polynomial control and the ideal control over a future time window $[0, T]$:

Optimization Objective

$$a_i(k) = \arg\min \int_0^T \left| f(a, \tau) - \hat{u}_i(\hat{X}, t_k + \tau) \right|^2 + \delta_i \left| f(a, \tau) \right|^2 \, d\tau$$

where:

- $f(a, \tau) = a_0 + a_1\tau + a_2\tau^2 + \cdots + a_p\tau^p$
- $\hat{u}_i(\hat{X},\, t_k + \tau)$ is the ideal control at future time
- δ_i is a design parameter

Interpretation of the Cost Function

This optimization has two key components:

1. **Tracking term**

$$|f(a, \tau) - \hat{u}_i(\cdot)|^2$$

2. **Regularization term**

$$\delta_i\, |f(a, \tau)|^2$$

This second term is important because without it, the polynomial could develop large coefficients, leading to oscillatory or unstable control behavior. The squared formulation also ensures smooth optimization and penalizes larger deviations more strongly.

Constraint and Its Implication

The optimization is subject to the constraint:

$$f(a, 0) = \hat{u}(t_k)$$

This ensures that at the start of the interval ($\tau = 0$), the polynomial matches the current control input. Since:

$$f(a, 0) = a_0$$

it directly follows that:

$$a_0 = \hat{u}(t_k)$$

Thus, the constant term is fixed, and only the remaining coefficients need to be optimized.

Convexity and Uniqueness

The optimization problem is strictly convex. This is because the associated Hessian matrix is positive definite. As a result:

- ✓ A **unique optimal solution** exists
- ✓ The coefficients can be computed efficiently
- ✓ The solution is stable and well-defined

22 Final Control Implementation

Once the coefficients are computed at time t_k, the control input applied by the robot is:

$$u_i(t_k + \tau) = \sum_{j=0}^{p} a_{ij}(k)\, \tau^j$$

for:

$$\tau \in [0,\, t_{k+1} - t_k), \qquad i \in \{1, 2\}$$

with:

$$u_1(t) = v(t), \qquad u_2(t) = \omega(t)$$

Key Insight

The entire method can be understood as performing a **least-squares fitting of the ideal control signal using a polynomial over time**. Instead of repeatedly sending control values, the controller sends a compact set of coefficients that define a function.

This allows the robot to:

- Compute control locally
- Follow a smooth trajectory
- Reduce communication significantly

In essence, the controller sends a control "recipe" rather than individual commands, enabling efficient and accurate trajectory tracking over a networked system.

23 Final Summary and Conclusion

This report has detailed the design and theoretical backing of **Event-Triggered Polynomial Control (ETPC)** for unicycle robots. By moving away from traditional Zero-Order Hold (ZOH) methods, we achieve a more sophisticated form of networked control.

Key Report Takeaways

- ✓ **Bandwidth Efficiency:** Polynomials allow for much longer intervals between communication events compared to constant-value ZOH.
- ✓ **Performance:** The robot maintains high-accuracy trajectory tracking despite significantly fewer updates.
- ✓ **Stability:** The system is proven to be **Uniformly Ultimately Bounded (UUB)**, ensuring the robot stays within a safe "box" around its target path.
- ✓ **Safety:** The math guarantees **Non-Zeno behavior**, meaning the hardware will never be overwhelmed by infinite rapid-fire updates.

— This concludes the technical report on ETPC Trajectory Tracking —